

Linear Quadratic Model Predictive Control: Notes from Theory to Praxis

Timothy Werner

Abstract. This is LQMPC – Linear-quadratic Model Predictive Control
This paper is still work in progres...

Keywords. MPC, IPM, Riccati

Contents

1	Introduction	1
2	The linear-quadratic OCP	2
3	Unconstrained linear-quadratic OCP	3
3.1	Solution via Equality Constraint Elimination	3
3.2	Solution via Karush-Kuhn-Tucker	4
3.2.1	From KKT to Block-Cholesky	5
3.2.2	From KKT to Dynamic Programming	6

1 Introduction

2 The linear-quadratic OCP

The finite-horizon linear-quadratic optimal control problem is formulated as follows.

$$\begin{aligned}
& \underset{x_k, u_k}{\text{minimize}} && \sum_{k=0}^N \frac{1}{2} \begin{bmatrix} x_k \\ u_k \\ 1 \end{bmatrix}^\top \begin{bmatrix} Q_k & S_k & q_k \\ S_k^\top & R_k & r_k \\ q_k^\top & r_k^\top & 0 \end{bmatrix} \begin{bmatrix} x_k \\ u_k \\ 1 \end{bmatrix} \\
& \text{subject to} && x_0 = \hat{x} \\
& && x_{k+1} = A_k x_k + B_k u_k + b_k, && k \in \mathcal{I}_0^{N-1} \\
& && \underline{x}_k \leq J_k^{bx} x_k \leq \bar{x}_k, && k \in \mathcal{I}_0^N \\
& && \underline{u}_k \leq J_k^{bu} u_k \leq \bar{u}_k, && k \in \mathcal{I}_0^N \\
& && C_k x_k + D_k u_k \leq d_k, && k \in \mathcal{I}_0^N
\end{aligned}$$

With the states $x_k \in \mathcal{X} \subseteq \mathbb{R}^{n_x}$ and inputs $u_k \in \mathcal{U} \subseteq \mathbb{R}^{n_u}$. The state and input dimensions are n_x and n_u , respectively. The time horizon is $N \in \mathbb{N}_{>0}$ and the initial state is given as $\hat{x} \in \mathcal{X}$. The system dynamics are described by the matrices $A_k \in \mathbb{R}^{n_x \times n_x}$, $B_k \in \mathbb{R}^{n_x \times n_u}$ and the vector $b_k \in \mathbb{R}^{n_x}$. The cost function is defined by the state weight matrices $Q_k \in \mathbb{S}_+^{n_x}$, which is required to be symmetric positive-semi definite. The input weight matrices R_k , which is required to be symmetric and positive-definite on the control subspace $R_k - S_k^\top Q_k^{-1} S_k \in \mathbb{S}_{++}^{n_u \times n_u}$, and the cross-term weight matrices $S_k \in \mathbb{R}^{n_u \times n_x}$. The linear terms of the cost function are given by the vectors $q_k \in \mathbb{R}^{n_x}$ and $r_k \in \mathbb{R}^{n_u}$. There are a number of n_{bx} box constraints on the states and n_{bu} box constraints on the inputs, which are mapped by the selection matrices $J_k^{bx} \in \mathbb{R}^{bx \times n_x}$ and $J_k^{bu} \in \mathbb{R}^{bu \times n_u}$, together with the lower bounds $\underline{x}_k \in \mathbb{R}^{bx}$, $\underline{u}_k \in \mathbb{R}^{bu}$ and upper bounds $\bar{x}_k \in \mathbb{R}^{bx}$, $\bar{u}_k \in \mathbb{R}^{bu}$ respectively. There are a number of n_p polytopic constraints represented by the matrices $C_k \in \mathbb{R}^{n_x \times n_p}$, $D_k \in \mathbb{R}^{n_u \times n_p}$ and the vector $d_k \in \mathbb{R}^{n_p}$. The integer set is denoted as $\mathcal{I}_a^b := \{a, a+1, \dots, b\}$.

3 Unconstrained linear-quadratic OCP

This section serves to explain the solution of the KKT system, which sits at the core of solving linear-quadratic optimal control problems. We will first consider the unconstrained case, to gain a better understanding of the KKT system and it's importance to the problem. The unconstrained finite-horizon linear-quadratic optimal control problem is formulated as follows.

$$\begin{aligned} & \underset{x_k, u_k}{\text{minimize}} && \sum_{k=0}^N \frac{1}{2} \begin{bmatrix} x_k \\ u_k \\ 1 \end{bmatrix}^\top \begin{bmatrix} Q_k & S_k & q_k \\ S_k^\top & R_k & r_k \\ q_k^\top & r_k^\top & 0 \end{bmatrix} \begin{bmatrix} x_k \\ u_k \\ 1 \end{bmatrix} \\ & \text{subject to} && x_0 = \hat{x} \\ & && x_{k+1} = A_k x_k + B_k u_k + b_k \end{aligned}$$

Note that *unconstrained* in this case means that there are no *inequality* constraints on states or inputs. The only constraints are the system dynamics and the initial state, but these are primal feasibility constraints, which means they must be satisfied in order to attain a feasible solution. They are thus not a constraint on the problem in the sense of restricting the solution space, rather an information to the problem about the underlying dynamics inherent to the system. Let's look at how we can compute a solution to this problem.

3.1. Solution via Equality Constraint Elimination. The optimization problem is formulated such that the decision variables are the state and input pairs (x_k, u_k) for all stages k over the time horizon N . Consecutive stages k and $k + 1$ are coupled, via the system dynamics in the form of the equality constraints

$$x_{k+1} = A_k x_k + B_k u_k + b_k, \forall k \in \mathcal{I}_0^{N-1}$$

This means that the state of a future stage x_{k+1} is completely determined by the state of the current stage x_k and the input of the current stage u_k . This iteration is bounded by the initial condition $x_0 = \hat{x}$ at the begin of the time horizon. This allows us to eliminate the states from the decision variables, leaving only the inputs at each stage as a decision variable, parametrized in the initial condition x_0 . To see this, let's explicitly calculate the state trajectory, starting at the initial stage.

$$x_0 = \hat{x}$$

The first stage is given by the first equality condition

$$x_1 = A_0 \hat{x} + B_0 u_0 + b_0$$

The second stage is given by the second equality condition

$$x_2 = A_1 x_1 + B_1 u_1 + b_1$$

We can now substitute the first stage into the second stage, eliminating x_1

$$\begin{aligned} x_2 &= A_1 x_1 + B_1 u_1 + b_1 \\ x_2 &= A_1 (A_0 \hat{x} + B_0 u_0 + b_0) + B_1 u_1 + b_1 \\ x_2 &= A_1 A_0 \hat{x} + A_1 B_0 u_0 + B_1 u_1 + A_1 b_0 + b_1 \end{aligned}$$

We can do this for each stage and thus eliminate all states x_k . The iteration can be written in the closed form for any given stage $k = j$ as

$$x_j = \prod_{k=0}^{j-1} A_k \hat{x} + \sum_{k=0}^{j-1} \prod_{i=k+1}^{j-1} A_i B_k u_k + \sum_{k=0}^{j-1} \prod_{i=k+1}^{j-1} A_i b_k$$

or in a less ugly and more readable form as vector matrix product

$$\mathbf{x} = \Phi \hat{x} + \Gamma \mathbf{u} + \Theta \mathbf{b}$$

where $\mathbf{x} = [x_1^\top, \dots, x_N^\top]^\top$, $\mathbf{u} = [u_0^\top, \dots, u_{N-1}^\top]^\top$, $\mathbf{b} = [b_0^\top, \dots, b_{N-1}^\top]^\top$ and

$$\begin{aligned} \Phi &= \begin{bmatrix} A_0 \\ A_1 A_0 \\ \vdots \\ \prod_{k=0}^{N-1} A_k \end{bmatrix} \\ \Gamma &= \begin{bmatrix} B_0 & 0 & \cdots & 0 \\ A_1 B_0 & B_1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ \prod_{i=1}^{N-1} A_i B_0 & \cdots & A_{N-1} B_{N-2} & B_{N-1} \end{bmatrix} \\ \Theta &= \begin{bmatrix} I & 0 & \cdots & 0 \\ A_1 & I & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ \prod_{i=1}^{N-1} A_i & \cdots & A_{N-1} & I \end{bmatrix} \end{aligned}$$

This expression can be substituted back into the objective, leaving a condensed QP in only inputs u_k , parametrized in the initial condition $\hat{x}$. Since the dimension of the problem has decreased from roughly $N * (n_x + n_u)$ to $N * n_u$, this process is known as *condensing*. The benefit of a smaller dimension, comes at the cost of a dense hessian. The hessian after the condensing is given as

$$\mathcal{H} = R + \Gamma^\top Q \Gamma$$

where $R := \text{diag}(R_k)$ and $Q := \text{diag}(Q_k)$. Since this paper focuses on the implementation of sparsity exploiting methods, this section is thus concluded at this point

3.2. Solution via Karush-Kuhn-Tucker. In terms of numeric optimization, the OCP above is an equality-constrained QP, which is convex in the decision variables x_k and u_k . It is convex, because the cost function is quadratic in x_k and u_k with $Q_k \in \mathbb{S}_+^{n_x}$ and $R_k - S_k^\top Q_k^{-1} S_k \in \mathbb{S}_{++}^{n_u \times n_u}$ per definition and the equality constraints are affine in x_k and u_k . Since the problem is convex, it has a unique solution. This solution is sufficiently characterized by the famous Karush-Kuhn-Tucker (KKT) conditions. To derive the KKT conditions, we first form the Lagrangian by introducing the Lagrange multipliers $\pi_k \in \mathbb{R}^{n_x}$ for the dynamic constraints.

$$\mathcal{L}(x_k, u_k, \pi_k) = \sum_{k=0}^{N-1} l_k(x_k, u_k) + l_N(x_N) + \sum_{k=0}^{N-1} \pi_{k+1}^\top (A_k x_k + B_k u_k + b_k - x_{k+1})$$

With the stage cost

$$l_k(x_k, u_k) = \frac{1}{2} x_k^\top Q_k x_k + \frac{1}{2} u_k^\top R_k u_k + x_k^\top S_k^\top u_k + q_k^\top x_k + r_k^\top u_k$$

and terminal cost

$$l_N(x_N) = \frac{1}{2} x_N^\top Q_N x_N + q_N^\top x_N$$

The KKT conditions are obtained by the stationary conditions, which say, that the Lagrangian becomes stationary at the optimal solution (x_k^*, u_k^*, π_k^*) with respect to all primal and dual variables.

$$\nabla \mathcal{L}(x_k^*, u_k^*, \pi_k^*) = 0$$

The stationary conditions are given as follows. Stationary w.r.t. states x_k for $k \in \mathcal{I}_0^{N-1}$:

$$\nabla_{x_k} \mathcal{L} = Q_k x_k + S_k u_k + q_k + A_k^\top \pi_{k+1} - \pi_k = 0$$

Stationary w.r.t. inputs u_k for $k \in \mathcal{I}_0^{N-1}$:

$$\nabla_{u_k} \mathcal{L} = R_k u_k + S_k^\top x_k + r_k + B_k^\top \pi_{k+1} = 0$$

Stationary w.r.t. Lagrange multipliers π_k for $k \in \mathcal{I}_1^N$:

$$\nabla_{\pi_k} \mathcal{L} = A_{k-1} x_{k-1} + B_{k-1} u_{k-1} + c_{k-1} - x_k = 0$$

If we stack the decision variables and Lagrange multipliers into a single vector, we can write the KKT conditions as a large linear system. This system is called the KKT-system. Here is an example showing the structure for $N = 2$.

$$\begin{bmatrix} Q_0 & S_0 & & & A_0^\top & \\ S_0^\top & R_0 & & & B_0^\top & \\ & & Q_1 & S_1 & -I & A_1^\top \\ & & S_1^\top & R_1 & & B_1^\top \\ & & & & Q_2 & -I \\ A_0 & B_0 & -I & & & \\ & & A_1 & B_1 & -I & \end{bmatrix} \begin{bmatrix} x_0 \\ u_0 \\ x_1 \\ u_1 \\ x_2 \\ \pi_0 \\ \pi_1 \end{bmatrix} = - \begin{bmatrix} q_0 \\ r_0 \\ q_1 \\ r_1 \\ q_N \\ b_0 \\ b_1 \end{bmatrix}$$

The system can be summarized in the following form, which is the standard system arising from optimal control problems.

$$\begin{bmatrix} \mathcal{H} & \mathcal{A}^\top \\ \mathcal{A} & 0 \end{bmatrix} \begin{bmatrix} y \\ \pi \end{bmatrix} = \begin{bmatrix} -h \\ -c \end{bmatrix}$$

Looking closely at this system, we can see that the Hessians $\mathcal{H}$ is block-diagonal and $\mathcal{A}$ is block-bidiagonal. That means the KKT system obeys a certain sparsity pattern. This pattern can be exploited for fast and reliable solutions of the system, which is the core idea of tailored LQMPC solvers and is the main focus of this paper. There are in general three approaches to solve this system. The first and naive approach would be to feed this system to a dense cholesky solver. This works, but finding the solution is of order $\mathcal{O}(N^3 \cdot (n_x + n_u)^3)$, i.e. cubic in the horizon and thus not relevant for practical applications. The other two approaches are worth considering in a bit more detail.

3.2.1. From KKT to Block-Cholesky. The second approach is to notice the *spacial* sparsity pattern of the hessian $\mathcal{H}$ and realize, the the solution to the system obayes a block-diagonal structure. To show this, let us attempt to solve this system by eliminating y from the KKT system, via the schur complement. We thus form

$$y = \mathcal{H}^{-1}(-h - \mathcal{C}^\top \pi)$$

and substitute into $\mathcal{C}y = -c$, from which we get

$$\mathcal{C}\mathcal{H}^{-1}\mathcal{C}^\top \pi = \mathcal{C}\mathcal{H}^{-1}h - c$$

We now define $Y := \mathcal{C}\mathcal{H}^{-1}\mathcal{C}^\top$ and $\beta := c - \mathcal{C}\mathcal{H}^{-1}h$, to arrive at the system in the so called normal equations form.

$$Y\pi = \beta$$

Let's closely observe the spacial structure of Y . We see that the matrix $\mathcal{H}$ is block-diagonal with the stage hessian blocks

$$H_k = \begin{bmatrix} Q_k & S_k \\ S_k^\top & R_k \end{bmatrix}$$

and the matrix $\mathcal{C}$ is block-bidiagonal. Thus Y has a block-tridiagonal structure in the following way.

$$Y = \begin{bmatrix} Y_{00} & Y_{01} & & \\ Y_{01}^\top & Y_{11} & Y_{12} & \\ & Y_{12}^\top & Y_{22} & \ddots \\ & & \ddots & \ddots \end{bmatrix}$$

with $\Phi_k := H_k^{-1}$, the blocks of Y are given as

$$Y_{k,k} = [A_{k-1} \quad B_{k-1}] \Phi_{k-1} \begin{bmatrix} A_{k-1}^\top \\ B_{k-1}^\top \end{bmatrix} + [I \quad 0] \Phi_k \begin{bmatrix} I \\ 0 \end{bmatrix}$$

$$Y_{k,k+1} = -[I \quad 0] \Phi_k \begin{bmatrix} A_k^\top \\ B_k^\top \end{bmatrix}$$

The next step is the cholesky factorization of Y

$$Y = L_Y L_Y^\top$$

Due to the block-tridiagonal structure of Y , L_Y is block-lower-bidiagonal:

$$L_Y = \begin{bmatrix} L_{00} & & & \\ L_{10} & L_{11} & & \\ & L_{21} & L_{22} & \\ & & \ddots & \ddots \end{bmatrix}$$

This fact motivates a blockwise factorization with the following recursion

$$L_{kk} L_{kk}^\top = Y_{kk} - L_{k,k-1} L_{k,k-1}^\top$$

$$L_{k+1,k} L_{kk}^\top = Y_{k+1,k}$$

Solve $L_Y z = \beta$ by forward substitution, solve $L_Y^\top \pi = z$ by backward substitution and finally, recover $y = H^{-1}(-h - C^\top \pi)$.

3.2.2. From KKT to Dynamic Programming. The third approach is to notice, that the problem also has temporal structure. To be fair, the spacial structure we exploited above also stems from a temporal structure, but the interpretation takes on a different route. Let's take a closer look at the KKT conditions to see the temporal relationship. The stationary conditions with respect to x_k can be rewritten as

$$\pi_k = Q_k x_k + S_k u_k + q_k + A_k^\top \pi_{k+1}$$

This reveals that the dynamics of the costates π_k propagate *backward* in time. It is bounded by the terminal condition $\pi_N = Q_N x_N + q_N$ at the end of the time horizon. The stationary conditions with respect to π_k reveal, that the dynamics of the states x_k propagate *forward* in time.

$$x_{k+1} = A_k x_k + B_k u_k + b_k$$

It is bounded by the initial condition $x_0 = \hat{x}$ at the begin of the time horizon. Both dynamics are coupled via the stationary conditions for the inputs u_k

$$R_k u_k + S_k^\top x_k + r_k + B_k^\top \pi_{k+1} = 0$$

These equations form a coupled two-point boundary value problem (BVP). They are not an initial value problem, which can be solved by forward integration. Nor are they a final value problem, which can be solved by backward integration. Suppose you try to solve it forward in time. You start at stage $k = 0$ with $x_0 = \hat{x}$. To compute u_0^* , you need π_1 . But π_1 depends on π_2 , which depends on π_3 , all the way to π_N . And π_N depends on x_N , which you don't know because you haven't simulated forward yet. Now suppose you try to solve it backwards. You start at stage $k = N$ with $\pi_N = Q_N x_N + q_N$. But again, you don't know x_N without having simulated forward from x_0 . The information flows in opposite directions, and they're coupled through the optimality condition for u_k . This is the essence of the two-point BVP. Data is specified at the boundaries of the time horizon, and you can't simply iterate through in one direction. The solution to this kind of problem comes from dynamic programming and Bellman's principle of optimality. Using the dynamic programming approach is a fundamental shift of perspective. Instead of searching for trajectories x^* , u^* and π^* that are characterized by the KKT conditions, you are searching for an optimal policy, i.e. a function $u_k^*(x_k)$ parametrized in the state x_k . This will decouple the two-point BVP and allows for solving it in one backward (policy calculation) and one forward (trajectory rollout) iteration. The key idea of dynamic programming is to reformulate the search for optimal *trajectories* into a search for an optimal *policy*, i.e. a function $u_k^*(x_k)$ that maps the current state to the optimal input. This decouples the two-point BVP and allows the solution to be computed in one backward pass (policy computation) followed by one forward pass (trajectory rollout).

Bellman's Principle of Optimality. Consider the total cost-to-go from stage k to the terminal stage N

$$J_k(x_k, u_k, \dots, u_{N-1}) = \sum_{j=k}^{N-1} l_j(x_j, u_j) + l_N(x_N)$$

subject to the dynamics $x_{j+1} = A_j x_j + B_j u_j + b_j$. The *value function* $V_k(x_k)$ is defined as the minimum of this cost-to-go over all admissible input sequences

$$V_k(x_k) = \min_{u_k, \dots, u_{N-1}} J_k(x_k, u_k, \dots, u_{N-1})$$

Bellman's principle of optimality states: *An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.* Applied to our problem, this yields the recursive relationship known as the *Bellman equation*

$$V_k(x_k) = \min_{u_k} \{l_k(x_k, u_k) + V_{k+1}(A_k x_k + B_k u_k + b_k)\}$$

with the terminal condition

$$V_N(x_N) = l_N(x_N) = \frac{1}{2} x_N^\top Q_N x_N + q_N^\top x_N$$

This equation is the foundation of the backward recursion. At each stage k , the minimization is only over the *current* input u_k . All future costs are already captured in the value function V_{k+1} . This is why dynamic programming breaks the curse of the coupled two-point BVP: you never need to know x_N explicitly. You only need to know V_{k+1} as a function of the state, and the dynamics do the rest.

Quadratic structure of the value function. Since the terminal cost is quadratic in x_N , and each stage adds quadratic terms and propagates through affine dynamics, we expect the value function to remain quadratic at every stage. We thus make the ansatz

$$V_k(x_k) = \frac{1}{2} x_k^\top P_k x_k + p_k^\top x_k + \eta_k$$

with $P_k \in \mathbb{S}_+^{n_x}$, $p_k \in \mathbb{R}^{n_x}$ and $\eta_k \in \mathbb{R}$. This ansatz will be verified by showing that if it holds at stage $k+1$, it also holds at stage k . The terminal condition gives

$$P_N = Q_N, \quad p_N = q_N, \quad \eta_N = 0$$

Expanding the Bellman equation. Substituting the ansatz for V_{k+1} and the stage cost l_k into the Bellman equation, we obtain the *action-value function* (or Q-function)

$$\mathcal{Q}_k(x_k, u_k) := l_k(x_k, u_k) + V_{k+1}(A_k x_k + B_k u_k + b_k)$$

such that $V_k(x_k) = \min_{u_k} \mathcal{Q}_k(x_k, u_k)$. Let us expand this expression term by term. The stage cost contributes

$$l_k(x_k, u_k) = \frac{1}{2} x_k^\top Q_k x_k + \frac{1}{2} u_k^\top R_k u_k + u_k^\top S_k^\top x_k + q_k^\top x_k + r_k^\top u_k$$

For the value function term, denote $\xi := A_k x_k + B_k u_k + b_k$. Then

$$V_{k+1}(\xi) = \frac{1}{2} \xi^\top P_{k+1} \xi + p_{k+1}^\top \xi + \eta_{k+1}$$

Expanding $\xi^\top P_{k+1} \xi$ gives

$$\begin{aligned} \xi^\top P_{k+1} \xi &= (A_k x_k + B_k u_k + b_k)^\top P_{k+1} (A_k x_k + B_k u_k + b_k) \\ &= x_k^\top A_k^\top P_{k+1} A_k x_k + u_k^\top B_k^\top P_{k+1} B_k u_k + b_k^\top P_{k+1} b_k \\ &\quad + 2 u_k^\top B_k^\top P_{k+1} A_k x_k + 2 b_k^\top P_{k+1} A_k x_k + 2 b_k^\top P_{k+1} B_k u_k \end{aligned}$$

and $p_{k+1}^\top \xi = p_{k+1}^\top A_k x_k + p_{k+1}^\top B_k u_k + p_{k+1}^\top b_k$. Now collect all terms in $\mathcal{Q}_k(x_k, u_k)$ according to their dependence on x_k and u_k . The result is again a quadratic form in (x_k, u_k)

$$\mathcal{Q}_k(x_k, u_k) = \frac{1}{2} \begin{bmatrix} x_k \\ u_k \\ 1 \end{bmatrix}^\top \begin{bmatrix} \tilde{Q}_k & \tilde{S}_k & \tilde{q}_k \\ \tilde{S}_k^\top & \tilde{R}_k & \tilde{r}_k \\ \tilde{q}_k^\top & \tilde{r}_k^\top & \tilde{\eta}_k \end{bmatrix} \begin{bmatrix} x_k \\ u_k \\ 1 \end{bmatrix}$$

with the *expanded stage cost* matrices

$$\tilde{Q}_k = Q_k + A_k^\top P_{k+1} A_k \tag{1}$$

$$\tilde{S}_k = S_k + A_k^\top P_{k+1} B_k \tag{2}$$

$$\tilde{R}_k = R_k + B_k^\top P_{k+1} B_k \tag{3}$$

$$\tilde{q}_k = q_k + A_k^\top (P_{k+1} b_k + p_{k+1}) \tag{4}$$

$$\tilde{r}_k = r_k + B_k^\top (P_{k+1} b_k + p_{k+1}) \tag{5}$$

Note the structure: these are the original stage cost matrices, augmented by the contribution of the future cost V_{k+1} propagated backward through the dynamics. The matrix $\tilde{Q}_k$ is the state cost after absorbing the future cost seen through A_k , $\tilde{R}_k$ is the input cost after absorbing the future cost seen through B_k , and $\tilde{S}_k$ captures the cross-coupling.

Minimization: the optimal policy. Since $\tilde{R}_k = R_k + B_k^\top P_{k+1} B_k$ inherits positive definiteness from $R_k \succ 0$ and $P_{k+1} \succeq 0$, the Q-function is strictly convex in u_k and the minimum is unique. Setting $\nabla_{u_k} \mathcal{Q}_k = 0$ yields

$$\tilde{R}_k u_k + \tilde{S}_k^\top x_k + \tilde{r}_k = 0$$

Solving for the optimal input gives

$$u_k^* = -\tilde{R}_k^{-1} \left(\tilde{S}_k^\top x_k + \tilde{r}_k \right) = K_k x_k + \kappa_k \quad (6)$$

where we define the *feedback gain* and *feedforward term*

$$K_k := -\tilde{R}_k^{-1} \tilde{S}_k^\top \quad (7)$$

$$\kappa_k := -\tilde{R}_k^{-1} \tilde{r}_k \quad (8)$$

This is the optimal policy we were looking for: an affine state-feedback law, parametrized at each stage by the gain K_k and the offset κ_k . The gain K_k is the *LQR gain* when $q_k = 0$, $r_k = 0$ and $b_k = 0$. The feedforward term κ_k accounts for the affine terms in the dynamics and the linear terms in the cost.

The Riccati recursion. Substituting the optimal policy (6) back into the Q-function and collecting terms in x_k , we verify the quadratic ansatz and obtain the recursion for P_k and p_k . The value function at stage k is

$$V_k(x_k) = \mathcal{Q}_k(x_k, u_k^*) = \frac{1}{2} x_k^\top P_k x_k + p_k^\top x_k + \eta_k$$

To obtain P_k , collect all terms quadratic in x_k from $\mathcal{Q}_k(x_k, K_k x_k + \kappa_k)$. The quadratic contribution from $u_k^* = K_k x_k + \kappa_k$ is

$$\frac{1}{2} x_k^\top \left(\tilde{Q}_k + K_k^\top \tilde{S}_k^\top + \tilde{S}_k K_k + K_k^\top \tilde{R}_k K_k \right) x_k$$

Substituting $K_k = -\tilde{R}_k^{-1} \tilde{S}_k^\top$ and simplifying (using $K_k^\top \tilde{R}_k K_k = \tilde{S}_k \tilde{R}_k^{-1} \tilde{S}_k^\top$ and $K_k^\top \tilde{S}_k^\top + \tilde{S}_k K_k = -2\tilde{S}_k \tilde{R}_k^{-1} \tilde{S}_k^\top$) yields

$$P_k = \tilde{Q}_k - \tilde{S}_k \tilde{R}_k^{-1} \tilde{S}_k^\top \quad (9)$$

This is the *discrete-time Riccati equation*. Expanding the shorthand (1)–(3), the fully explicit form reads

$$P_k = Q_k + A_k^\top P_{k+1} A_k - (S_k + A_k^\top P_{k+1} B_k)(R_k + B_k^\top P_{k+1} B_k)^{-1} (S_k + A_k^\top P_{k+1} B_k)^\top$$

Similarly, collecting all terms linear in x_k gives

$$p_k = \tilde{q}_k - \tilde{S}_k \tilde{R}_k^{-1} \tilde{r}_k \quad (10)$$

and the constant term (which is not needed for computing the optimal trajectory, but completes the recursion) is

$$\eta_k = \tilde{\eta}_k - \frac{1}{2} \tilde{r}_k^\top \tilde{R}_k^{-1} \tilde{r}_k + \eta_{k+1}$$

Equations (9) and (10) together with the definitions (1)–(5) and the terminal conditions $P_N = Q_N$, $p_N = q_N$ form the *backward Riccati recursion*. Note the Schur complement structure: P_k is obtained from the expanded stage Hessian by eliminating the input block via the Schur complement with respect to $\tilde{R}_k$. This is not a coincidence. It is the matrix-level manifestation of the minimization over u_k .

Connection to the costates. The value function gradient with respect to the state is

$$\nabla_{x_k} V_k(x_k) = P_k x_k + p_k$$

We now show that this equals the costate π_k from the KKT conditions. Recall the costate dynamics

$$\pi_k = Q_k x_k + S_k u_k + q_k + A_k^\top \pi_{k+1}$$

If we define $\pi_k := P_k x_k + p_k$ and substitute $u_k^* = K_k x_k + \kappa_k$ and $\pi_{k+1} = P_{k+1} x_{k+1} + p_{k+1}$ with $x_{k+1} = A_k x_k + B_k u_k^* + b_k$, the right-hand side becomes

$$\begin{aligned} & Q_k x_k + S_k (K_k x_k + \kappa_k) + q_k + A_k^\top [P_{k+1} (A_k x_k + B_k (K_k x_k + \kappa_k) + b_k) + p_{k+1}] \\ &= [Q_k + S_k K_k + A_k^\top P_{k+1} (A_k + B_k K_k)] x_k \\ & \quad + [S_k \kappa_k + q_k + A_k^\top P_{k+1} (B_k \kappa_k + b_k) + A_k^\top p_{k+1}] \end{aligned}$$

Using the definitions of $\tilde{Q}_k$, $\tilde{S}_k$ and $K_k = -\tilde{R}_k^{-1} \tilde{S}_k^\top$, the coefficient of x_k simplifies to

$$\begin{aligned} Q_k + S_k K_k + A_k^\top P_{k+1} (A_k + B_k K_k) &= \tilde{Q}_k + \tilde{S}_k K_k \\ &= \tilde{Q}_k - \tilde{S}_k \tilde{R}_k^{-1} \tilde{S}_k^\top \\ &= P_k \end{aligned}$$

and the constant part simplifies to p_k by an analogous calculation. Thus $\pi_k = P_k x_k + p_k = \nabla_{x_k} V_k(x_k)$, which confirms the classical result: *the costate is the gradient of the value function*. This is the link between the KKT perspective and the dynamic programming perspective. The Riccati recursion computes the value function backward in time, which implicitly solves the backward costate dynamics. The optimal policy $u_k^* = K_k x_k + \kappa_k$ then provides the missing coupling that decouples the two-point BVP into two sequential sweeps.

The complete algorithm. Combining the backward and forward passes, the solution of the unconstrained LQ-OCF via the Riccati recursion proceeds as follows.

Backward pass ($k = N-1, N-2, \dots, 0$): Given $P_N = Q_N$, $p_N = q_N$, compute for each stage

$$\begin{aligned} \tilde{Q}_k &= Q_k + A_k^\top P_{k+1} A_k \\ \tilde{S}_k &= S_k + A_k^\top P_{k+1} B_k \\ \tilde{R}_k &= R_k + B_k^\top P_{k+1} B_k \\ \tilde{q}_k &= q_k + A_k^\top (P_{k+1} b_k + p_{k+1}) \\ \tilde{r}_k &= r_k + B_k^\top (P_{k+1} b_k + p_{k+1}) \\ K_k &= -\tilde{R}_k^{-1} \tilde{S}_k^\top \\ \kappa_k &= -\tilde{R}_k^{-1} \tilde{r}_k \\ P_k &= \tilde{Q}_k - \tilde{S}_k \tilde{R}_k^{-1} \tilde{S}_k^\top \\ p_k &= \tilde{q}_k - \tilde{S}_k \tilde{R}_k^{-1} \tilde{r}_k \end{aligned}$$

Forward pass ($k = 0, 1, \dots, N-1$): Given $x_0 = \hat{x}$, roll out the trajectory

$$\begin{aligned} u_k^* &= K_k x_k + \kappa_k \\ x_{k+1} &= A_k x_k + B_k u_k^* + b_k \end{aligned}$$

If the costates are needed (e.g. for constraint handling in an interior point method), they are recovered as $\pi_k = P_k x_k + p_k$. The computational cost of this procedure is $\mathcal{O}(N \cdot (n_x + n_u)^3)$, i.e. *linear* in the horizon length N and cubic in the state and input dimensions. The cubic cost per stage is dominated by the inversion of $\tilde{R}_k \in \mathbb{S}_{++}^{n_u}$ (or equivalently, its Cholesky factorization) and the Schur complement products.